

HDOC – Day 4/5

Multiply two integers without using *operator

Question

The following problem is to find the product of two integers without using the multiplication operator *. The given algorithm and flow chart are incorrect. Study them and complete the following tasks using repeated addition.

The solution must use the minimum possible number of additions and correctly handle positive x positive , positive x negative, negative x positive, negative x negative ,and multiplication by zero.

Tasks

1. Identify the mistakes in the given algorithm.
2. Rewrite the corrected algorithm
3. Identify the mistakes in the given flowchart
4. Redraw the corrected flowchart
5. Verify the corrected algorithm and flowchart using the given test cases.

1. Identify the Mistakes in the Given Algorithm

Given Algorithm

1. Define num 1 ,num2, prod,index as integers.
2. Get num1,num 2.
3. Set prod=0, index = 1
4. Repeat steps 4.1 and 4.2 until (index <= num2)
 - 4.1 prod=prod+num1
 - 4.2 index = index +1
5. Print prod.
6. Stop.

Mistakes --

1. The algorithm always uses num2 as the repetition count, so it does not guarantee the minimum number of additions.
2. It does not compare the absolute values of num1 and num2 to determine which value should be used as the repetition count.
3. The condition $\text{index} \leq \text{num2}$ does not work correctly when num2 is negative.
4. It does not correctly handle all combinations of positive and negative integers.
5. It does not separately determine whether the final product should be positive or negative.
6. Multiplication by zero is not explicitly handled. If either number is zero, the result should be 0 with 0 additions.

2. Corrected Algorithm

Algorithm : Multiplication Using Minimum repeated additions

1. Start

2. Input num1 and num2

3. Set prod = 0

4. If num1 = 0 OR num2 = 0, print and go to step 12.

5. If num1 and num2 have the same sign, set sign = 1; otherwise set sign = -1.

6 Set:

a = |num1|

b = |num2|

7. If a ≤ b:

- Set count = a
- Set value = b

Otherwise:

- Set count=b
- Set value =a

8.set index = 1

9.while index <=count:

- prod = prod + value
- index = index + 1

10.If sign = -1, change the sign of prod.

11.Print prod.

12.Stop.

Explanation

The smaller absolute value is used as the repetition count and the larger absolute value is repeatedly added. Therefore, the number of additions is minimized.

For example:

$12 \times 3 \rightarrow 12 + 12 + 12 = 36 \rightarrow 3$ additions

instead of adding 3 twelve times.

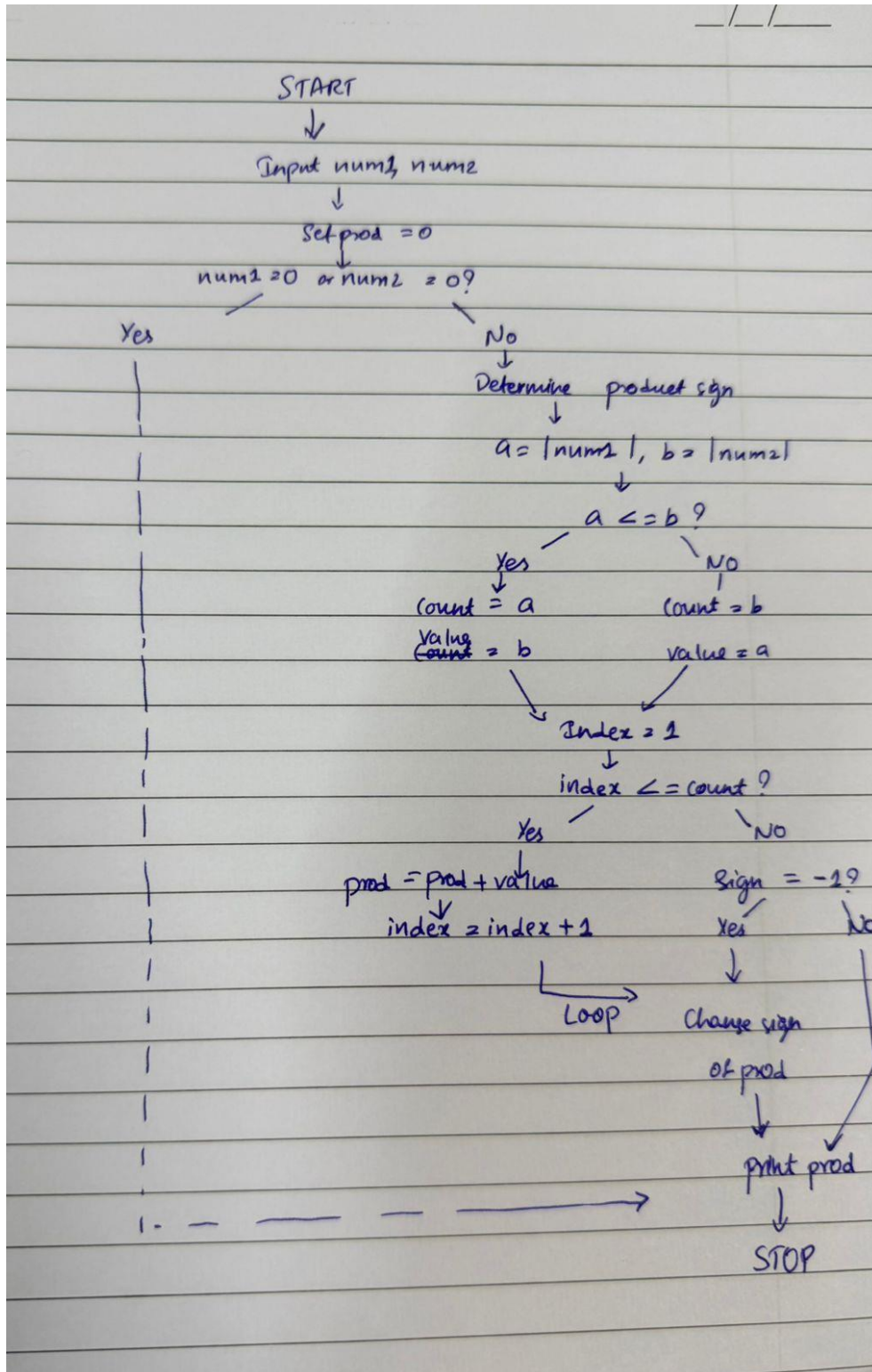
This is necessary because the supplied test cases explicitly require 12×3 and 3×12 to both use only 3 additions.

3. Identify the Mistakes in the Given Flowchart

Mistakes--

- 1.The flowchart directly uses num2 as the loop limit without considering whether it is positive or negative.
- 2.It does not compare $|\text{num1}|$ and $|\text{num2}|$ to choose the smaller value as the repetition count.
- 3.Therefore, it does not guarantee the minimum number of additions.
- 4.There is no decision to check whether either input is zero.
- 5.There is no logic to determine whether the final product should be positive or negative.
- 6.It does not correctly handle negative inputs.
- 7.The decision $\text{index} \leq \text{num2}$ is insufficient for all the required test cases.
- 8.The flowchart needs separate processing for absolute values, sign determination, selection of the repetition count, and application of the final sign.

4. Corrected Flowchart



5. Verification Using the Given Test Cases

Test	Input	Repeated Addition / Verification	Product	Additions	Result
1	12, 3	$12 + 12 + 12$	36	3	pass
2	3, 12	$12 + 12 + 12$	36	3	pass
3	-6, 5	$6 + 6 + 6 + 6 + 6 = 30$, apply – sign	-30	5	pass
4	6, -5	$6 + 6 + 6 + 6 + 6 = 30$, apply – sign	-30	5	pass
5	-9, -4	$9 + 9 + 9 + 9$	36	4	pass
6	-4, -9	$9 + 9 + 9 + 9$	36	4	pass
7	0, 8	Zero detected; no addition required	0	0	pass
8	8, 0	Zero detected; no addition required	0	0	pass
9	0, -7	Zero detected; no addition required	0	0	pass
10	-7, 0	Zero detected; no addition required	0	0	pass
11	0, 0	Zero detected; no addition required	0	0	pass
12	1, -7	7, apply – sign	-7	1	pass
13	-1, -9	9	9	1	pass

Conclusion:

The corrected algorithm and flowchart correctly handle positive, negative and zero inputs. By using the smaller absolute value as the repetition count, unnecessary additions are avoided. **All 13 given test cases pass with the required minimum number of additions.**